

GITHUB ENGINEERING INTELLIGENCE SYSTEM

Analyser Architecture, Scorecard Design & Employer Usability

Version 5.0 · May 2026 · CTO Office · Engineering Hiring Intelligence

2 ANALYSIS MODES
 Light + Deep

7 PRIMITIVES
 Canonical Dimensions

EVIDENCE BRIEF
 No Composite Score

ZERO SETUP
 Light Mode Instant

1. Philosophy & Design Goals

The GitHub Engineering Intelligence System is an evidence-based analyser of observable engineering signals. It is designed as a force-multiplier for engineering leaders — not an automated hiring decision-maker. Its output is a structured Evidence Brief grounded in verifiable signals, honest about what it cannot observe, and organised around seven durable assessment primitives that map directly to on-the-job performance.

This document describes the full architecture of the analyser itself: how it collects data, what it measures, how it presents findings, and how companies interact with it. It does not cover downstream hiring workflows, offer management, or candidate onboarding — those processes remain with the hiring organisation.

1.1 Core Design Principles

Principle	What It Means in Practice
Evidence over scores	The primary output is an evidence brief — specific, cited, traceable — not a composite number. Scores are a summary of evidence, never a replacement for it.
Honest about uncertainty	Every assessed dimension carries a confidence level. Dimensions that cannot be assessed are surfaced explicitly as observability gaps, never as penalties.
Two analysis modes	A light public-signal mode requires no candidate action and starts instantly. A deep authenticated mode unlocks private signals with candidate consent. Both modes are meaningful on their own.
Leverage existing tools	Where mature open-source or third-party tools already solve a problem reliably, the system delegates to them rather than reimplementing. This reduces maintenance burden, accelerates development, and improves reliability.
Seven canonical primitives	All signals, layers, and output sections map back to one of seven durable assessment dimensions. Nothing that does not contribute to a primitive belongs in the system.
Frictionless for companies	Employers need no integration, no training, and no configuration to begin. Light Mode delivers an immediate brief from a GitHub

Principle	What It Means in Practice
	username alone.
Activity does not equal ability	GitHub signals are among the best available proxies for engineering behaviour — but they are proxies. The system never conflates observable GitHub output with engineering intelligence or potential.
AI use is a competency, not a red flag	Effective AI leverage is a primary engineering skill in 2026. The system classifies how a candidate uses AI — not whether they use it.

1.2
 What This System Assesses and What It Does Not

The following qualities can be assessed from observable engineering signals and are within the scope of this system:

- Execution reliability: shipping consistency, CI discipline, commit quality
- Systems evolution: whether codebases improve under the engineer's stewardship
- Collaboration leverage: review quality, mentoring signals, cross-team engagement
- Technical depth: substantive architectural decisions, operational maturity indicators
- AI leverage quality: velocity-to-quality ratio, AI-generated vs. AI-guided code patterns
- Authenticity confidence: employment claim cross-referencing, gaming pattern detection

The following qualities are categorically outside the system's epistemic reach regardless of signal sophistication. They must be assessed by humans in a technical interview:

- Creativity and novel problem framing
- Systems intuition — sensing failure modes before they manifest
- Leadership under ambiguity and organisational change
- Product judgment — knowing what to build, not just how
- Learning velocity across genuinely new capability domains
- Founder energy and intrinsic motivation
- Cultural fit and team chemistry

DESIGN
MANDATE

Every signal added to the system must answer two questions: (1) Does it correlate with actual engineering performance, or with the appearance of performance? (2) What does it cost a motivated candidate to fake it, and does that cost exceed the benefit of gaming the system? Signals that fail either test are not included.

2. Two Analysis Modes

The system operates in two distinct modes that differ in access level, depth of analysis, and required setup. Both modes produce an Evidence Brief using the same seven-primitive framework; the difference is in the richness of signals available and the confidence levels achievable.

	Mode 1 — Light Analysis	Mode 2 — Deep Analysis
Trigger	Employer pastes a GitHub username or username list into the dashboard. No candidate action required.	Employer sends a candidate an evaluation link. Candidate authenticates via GitHub OAuth and installs the GitHub App on their account.
GitHub token	Platform system token — our shared GitHub App token. No per-candidate credential.	Candidate-specific installation token generated per GitHub App installation. Scoped to that candidate's repos and orgs.
Data access	Public repositories, public contribution activity, public PR/issue data, public profile, package registry presence, contribution calendar (public events only).	All public data plus: private repositories (with explicit consent), organisation membership and repos, full commit history, private PR review history, CI/CD run history, code scanning alerts, Dependabot alerts.
Local tool execution	Not performed. Analysis is API-only — no cloning. Faster, no compute overhead.	Full tool suite: scc, tokei, gitinspector, gitleaks, semgrep (lightweight ruleset). Top 30 repos cloned and analysed in parallel across 4 workers.
Primitives covered	All 7 primitives addressed at moderate-to-low confidence. Observability gaps are frequent and expected — surfaced explicitly.	All 7 primitives addressed. Higher confidence across the board. Fewer observability gaps. Private-work evidence significantly improves Authenticity Confidence.
Time to brief	Under 3 minutes for a typical public profile (50 repos, 5 years of history).	8–15 minutes depending on repository count and size. Runs asynchronously — employer is notified on completion.
Best used for	Initial screening of inbound applications. Rapid signal check before committing to a full evaluation or outreach.	Candidates who have passed initial screening and are being seriously considered. Pre-interview preparation. Full evidence gathering.
Rate limits	Platform token: 5,000 REST req/hr shared across all concurrent Light Mode runs. GraphQL: 5,000 points/hr. Search API: 30 req/min. Circuit breaker activates at <500 remaining.	Candidate installation token: 5,000 REST req/hr per candidate — fully isolated. No impact on Light Mode pool. GraphQL: 5,000 points/hr per installation.

IMPORTANT

Both modes operate under read-only permissions at all times. No data is written to GitHub. No source code is stored beyond the analysis session. Only derived metrics and signals are persisted. Private repo code is analysed in-memory only during the Deep Mode session.

8. Evidence Brief — Analyser Output Format

The Evidence Brief is the primary output of the analyser. It is not a score, a ranking, or a recommendation — it is a structured account of what the system observed, what it concluded, how confident it is in each conclusion, and where human judgment must take over. The Brief is structured around the seven canonical primitives. Every claim is traceable to a specific signal, a specific data source, and a confidence level.

CRITICAL DESIGN PRINCIPLE

The system NEVER outputs a composite score. A number hides the evidence behind it, suppresses uncertainty, and invites decisions based on the number rather than the reasoning. The Evidence Brief is designed to make the hiring manager more informed, not to make a decision for them.

8.1 Brief Structure

Section	Content	Purpose
A — Profile in 90 Seconds	Engineering operating style archetype (Production Engineer / OSS Contributor / Generalist Builder / Specialist / Ops-Focused / Research-Oriented). Three strongest evidenced capabilities with specific citation. Employment verification rung achieved per claim. AI Leverage Quality classification. Analysis mode used (Light / Deep). Recommended interview depth.	Scannable by a recruiter or CTO in under a minute. Answers: who is this person, can I trust the evidence, how deep do we need to go?
B — Tech Reality vs. CV Claims	Languages: claimed vs. evidenced by actual commit volume over the last 3 years — not repo count, not self-reported. Frameworks: mentioned vs. demonstrated with meaningful usage depth in actual code. Infrastructure: claims vs. evidence of IaC, ops tooling, actual deployments. Explicit flags for CV claims with zero corroborating evidence.	Directly replaces the most-misrepresented section of any CV. Gives hiring managers a verifiable account before the first call.
C — Work Pattern Intelligence	Shipping velocity: time from issue creation to merged PR, averaged over career timeline. Quality discipline trajectory over time. Collaboration style: does the candidate seek review or avoid it? AI Leverage Quality classification with supporting evidence. Communication quality in PR descriptions and code comments.	Answers questions a CV structurally cannot answer: how does this person actually work, not just what have they built.
D — Red Flags & Verification Gaps	All gaming flags and verification gaps listed with: the specific evidence that triggered the flag, the confidence level (how certain is this a flag vs. a false positive), the recommended interview question to surface or clear the flag, and whether it is a hard stop or a soft concern requiring follow-up.	Gaming signals are never buried in score arithmetic. Every flag is visible, traceable, and actionable — with a clear path to resolution via interview.
E — Interview Intelligence	3–5 technical questions generated from actual design decisions observed in the candidate's code. Questions probing gaps between CV claims and evidenced capabilities. Questions that explore red flags without revealing the detection mechanism (allowing the candidate to either confirm or clear it). Suggested interviewer pairing recommendation.	Transforms the first technical conversation from generic assessment to targeted, evidence-grounded investigation.
F — Role & Stack Match	For Deep Mode and when a JD is provided: technical overlap between candidate's evidenced stack and the role requirements. Gap analysis: skills required by the role with no evidence in the	Tells the hiring manager specifically where to probe and what to not bother probing because evidence is already

Section	Content	Purpose
	candidate's profile — translated into specific interview probe topics. JD intent extraction: what the role actually needs, beyond keyword matching.	strong.
G — What This Evaluation Cannot Tell You	Explicit list of the qualities within the epistemic boundary of the system — routed to specific interview probes. This section is permanent and never omitted. It tells the hiring manager exactly where human judgment must take over.	Trust builder. Makes the system more credible by being honest about its limits before someone discovers them.

8.2 Confidence Levels — Mandatory Language

Every assessed dimension in the Evidence Brief carries a confidence level. The following language is mandatory — implementers may not substitute different formulations, as variance in confidence language leads to misinterpretation.

Level	Trigger Condition	Mandatory Output Language
Strong Evidence	3 or more independent signals confirming the same capability across 12 or more months of activity	"Demonstrated across [N] repositories and [N] months — high confidence."
Moderate Evidence	1–2 signals or a single time window of evidence	"Evidenced in limited context — probe in interview to confirm depth."
Low Evidence	Single weak signal or isolated instance	"One instance detected — insufficient to score. Treat as unconfirmed in hiring decision."
Observability Gap	Signal is expected for a candidate of this seniority or role, but is absent or unverifiable (likely due to private enterprise work, classified context, or a non-GitHub workflow)	"No public evidence — likely private or enterprise context. Do not penalise. Recommend: [specific interview question to probe the capability directly]."
Insufficient Data (Profile-Level)	Majority of primary primitives return Observability Gap — typically senior enterprise or regulated-industry engineers	"This profile cannot be assessed from available public signals. Do not use this report as a filter for this candidate. Proceed directly to technical interview using the generated interview questions."

PROFILE-LEVEL GATE

When a senior-level candidate (Staff or above based on CV claims) produces a profile-level Insufficient Data output, the Evidence Brief must include: 'This profile pattern is consistent with enterprise or regulated-industry engineering contexts where public evidence is structurally absent. This is correlated with — not anticorrelated with — seniority and impact. Proceed to technical interview.'

9. Employer Dashboard & Usability

The employer-facing interface is designed around a single principle: zero mandatory configuration, zero friction to first value. An engineering leader should be able to evaluate a candidate from a GitHub username in under five minutes, with no integration, no training, and no setup. All advanced features are opt-in, not required.

9.1 Light Mode — Zero-Setup Flow

Step	Employer Experience	What Happens Behind the Scenes
1	Paste one or more GitHub usernames into the search bar (or upload a CSV). Select target role and seniority level from dropdown menus. Click Analyse.	System queues Light Mode evaluation jobs. Platform token initialised. Rate limit budget calculated.
2	Status indicator shows 'Analysing'. No further action required. Email notification optional.	API crawl of public data: Groups A, B (partial), F. GraphQL bulk fetch. Anti-gaming signals computed via GitHub Search API.
3	Evidence Brief appears in dashboard. Section A visible immediately — full brief below.	Brief generated. Confidence levels assigned. LLM API called for NLP analysis of commit messages, PR descriptions, AI pattern detection.
4	Employer reads the brief. Sections B–G available on single scroll. No navigation required.	All data local to the brief — no additional API calls on view.

9.2 Deep Mode — Candidate Evaluation Flow

Step	Employer Experience	Candidate Experience
1	Select candidate from dashboard. Click 'Request Deep Evaluation'. System generates a unique evaluation link.	Receives evaluation link via email (or directly from employer).
2	Dashboard shows 'Awaiting candidate authorisation' status.	Clicks link. Sees a clear, plain-language explanation of what access is being requested and why. Selects which repositories and organisations to include. Installs GitHub App.
3	Dashboard transitions to 'Evaluation in progress'. Estimated completion time shown. Email notification on completion.	No further action required. Receives a confirmation with a summary of what was accessed.
4	Full Evidence Brief available in dashboard. Deeper signal confidence vs. Light Mode brief clearly indicated per section.	N/A

9.3 Candidate Pull — Not Candidate Push

The verification burden is designed to be pulled by the system, not pushed onto the candidate. The system verifies as much as possible from public signals before requesting any candidate action. When candidate action would materially improve the evidence quality — and therefore improve the candidate's own evaluation — the framing is: 'We found X from your public profile. By connecting your GitHub account, we can also include Y and Z, which typically improves evidence confidence significantly.'

Principle	Wrong Framing (Push)	Correct Framing (Pull)
Verification	"Submit account links and employment verification materials to complete your evaluation."	"We've confirmed your @company.com commit history (Rung 3). Your private repos and org activity will further strengthen evidence confidence if you choose to connect them."
Gaps	"Your profile is missing evidence for [X]. Please explain."	System surfaces gaps to the hiring manager as specific interview questions. Candidate is not burdened with explaining the system's limitations.
Deep Mode	"You must complete a GitHub App installation to proceed."	"Your public profile is analysed. Connecting GitHub will add private repo signals — typically improving 3–4 primitive confidence levels significantly."

9.4 Outcome Feedback — Hire / No-Hire

The system collects a single feedback signal per candidate: the hire/no-hire outcome. This is the starting fuel for outcome calibration and is the only feedback mechanism that is frictionless enough to generate consistent data.

Feedback Mechanism	Detail	Why This, Not More
ATS webhook (preferred)	If the employer's Applicant Tracking System supports webhooks, a hire/no-hire event is captured automatically when the candidate's status changes. No manual action required from the hiring manager.	Zero friction. Data arrives without anyone having to remember to submit it. Produces consistent, unbiased signal.
Single-click dashboard button (fallback)	If no ATS integration is available: a binary 'Hired / Not Hired' button appears in the dashboard after evaluation. One click, no form.	One click is the maximum acceptable friction for feedback that busy hiring managers will actually provide.
No 90-day check-ins	Post-hire performance forms are not requested. Hiring managers will not complete them consistently enough to produce calibration-grade data — they will over-index on spectacular outcomes and ignore the middle ground.	Data from voluntary check-in forms suffers from extreme survivorship bias. The hire/no-hire signal is imperfect but consistent and unbiased.
No HRIS integration	The system does not request or require integration with HR information systems (Workday, BambooHR, Rippling) for performance or retention data. These integrations are governed by employment law and corporate policy — not achievable at scale for a hiring tool.	This class of integration takes years of enterprise sales and legal negotiation even when companies are willing. It is not a near-term data source.

10. Job Description Matching & Role Benchmarks

Matching candidates to roles is a two-tier system. The default tier requires no employer integration and is available from Phase 1. The premium tier — matching against the hiring team's actual codebase — is an opt-in feature available for enterprise customers willing to grant repository read access under a formal data processing agreement.

Tier	Mechanism	Availability	Integration Required
Default — JD Signal Extraction	Natural language processing (via LLM API) extracts actual technical requirements from the job description — not keyword matching, but intent extraction. 'Experience with high-throughput data pipelines' maps to specific observable signals in the candidate's Evidence Brief. The JD is treated as a noisy but available proxy for the team's needs.	Phase 1 — all customers	None — paste or upload JD text
Standard — Role Benchmark Profiles	Pre-built benchmark profiles per role archetype (Backend, Platform, ML, Security, etc.), derived from aggregated signal patterns across all candidates evaluated by the system. These profiles become richer and more accurate over time as evaluation history grows — without requiring any single company to share private data.	Phase 2 — built from evaluation history	None
Premium — Team Codebase Profile	Hiring team grants read access to their main repositories via the same GitHub App mechanism used for candidates. The system computes a tech fingerprint of the team's actual codebase: languages by commit volume, frameworks in active use, infrastructure patterns, test philosophy, CI/CD sophistication. Every candidate comparison then shows specific overlap and gap analysis relative to the team's real work.	Phase 3 — enterprise tier only, opt-in, requires security review and DPA	GitHub App installation on employer's org — opt-in, with explicit security review and a signed data processing agreement

IMPORTANT
— TEAM
CODEBASE
ACCESS

The same security considerations that prevent candidates from sharing private code will prevent CTOs from granting an external service access to their company's core infrastructure. The Team Codebase Profile is positioned as an opt-in premium feature — never as a default expectation or a required integration. Companies that choose to opt in receive materially richer matching; those that do not receive JD Signal Extraction and Role Benchmarks, which are independently valuable.

13. Epistemic Boundaries — What This System Cannot Claim

Every capability system that iterates without discipline drifts toward overclaiming. The following boundaries are permanent — they reflect the categorical limits of what observable engineering signals can assess, not temporary gaps to be closed by future development. They are not a disclaimer. They are a service to hiring managers: an honest map of where human judgment must take over, with suggested interview routing for each unclaimed dimension.

Dimension	Why It Cannot Be Assessed From Observable Signals	Interview Routing Suggestion
Creativity & novel problem framing	Observable code is the output of creative work, not the process. The most creative engineers sometimes produce the simplest solutions — indistinguishable in signal terms from unremarkable solutions.	Present a genuinely novel problem with no obvious prior-art solution. Watch whether they seek analogies, question the framing, or immediately pattern-match.
Systems intuition	Sensing failure modes before they manifest is fundamentally invisible in historical code. Code that never had a bug looks identical to code that was written defensively to avoid one.	Walk through a non-trivial architecture diagram and ask what they would be worried about after six months in production.
Leadership under ambiguity	Leadership is a relationship property. It cannot be observed in solo repositories or even in PR review depth without the surrounding organisational context.	Ask for a specific situation where they had to make a consequential technical decision with insufficient information. Focus on the decision process, not the outcome.
Product judgment	Knowing what to build requires understanding of user needs, business context, and strategic tradeoffs — none of which are observable in a GitHub commit history.	Present a real product decision the company has recently faced or is facing. Ask how they would approach the trade-off.
Learning velocity	Acquiring genuinely new capability is definitionally not visible in historical work. Repositories show what was learned, not how fast new things are being learned.	Give them a technical domain they have no evidenced experience in. Ask them to explain the core trade-offs in that domain as far as they can reason through. Watch the quality of their reasoning under uncertainty.
Adaptability under organisational change	Observable code reflects technical context, not organisational context. Resilience and adaptability are entirely invisible to this system.	Ask for a specific example of a significant organisational or technical pivot they worked through. Focus on how they navigated the transition, not whether it succeeded.
Founder energy & intrinsic motivation	Motivation is a psychological property. Observable output patterns can suggest consistency, but not the source of that consistency.	Ask what they are working on outside of their current role — not as a signal itself, but as a conversation starter about what drives them technically.
Cultural fit & team chemistry	Chemistry is relational and contextual. No signal model can predict whether this person will thrive in a specific team's dynamic.	Arrange a working session — even 30 minutes solving a real problem with two current team members. Observe interaction, not output.

GOVERNING PRINCIPLE

Every observability gap is a limitation of the system, not a limitation of the candidate. Every claim the system makes must be traceable to a primitive, grounded in evidence, and bounded by an explicit epistemic limit. Any signal, layer, or output that fails both tests does not belong in this system.

SYSTEM DESIGN SUMMARY

The GitHub Engineering Intelligence System produces a more rigorous, evidence-grounded, and uncertainty-honest assessment of an engineer's observable work history than any manual review could achieve in a reasonable time budget.

It does this through two analysis modes — a zero-friction Light Mode that works from a GitHub username alone, and a Deep Mode that unlocks private signals with candidate consent — both organised around seven durable canonical primitives that map directly to on-the-job engineering performance.

It delegates to mature external tools wherever they solve a problem better than a custom build would: GitHub's own search index for code similarity detection, third-party plagiarism APIs for laundering confirmation, an LLM API for all natural language analysis. It does not maintain custom indices, does not require HRIS integrations, and does not ask candidates to risk their current employment to participate.

Its output is an Evidence Brief — not a score — that tells hiring managers what was observed, how confident the system is in each conclusion, and precisely where human judgment must take over. Its governing principle is honesty: every observability gap is a limitation of the system, not a limitation of the candidate.